AgentMPI: A Message Passing Interface for Multi-Agent Harness Development

Abstract

Multi-agent systems built on large language models are written the way parallel programs were written before 1994: every project invents its own communication mechanism, and none of them compose. We argue that the field needs what MPI gave parallel computing — not a framework, but a *library-level protocol* with scoped communication contexts, typed transfers, collective operations with explicit algorithms, one-sided shared state with epochs, and failure mitigation — and we show that MPI's abstractions transfer with three specific, load-bearing modifications.

First, the receiving agent's context window is a scarce, overcommitted buffer, so the eager/rendezvous distinction that MPI hides as an implementation detail must become semantically visible; making it visible turns MPI's notion of an *unsafe program* into a precise and checkable notion of *context safety*. Second, reduction operators implemented by a language model are not associative and are *lossy in proportion to the depth of the reduction*, which makes collective-algorithm selection a quality decision rather than only a cost decision — with the surprising consequence that the logarithmic-depth tree wins on fidelity as well as latency, provided broadcasts forward immutable content-addressed handles rather than regenerated text. Third, the dominant agent failure is not the crash MPI's fault-tolerance work assumes but a well-formed, confident, wrong answer; this is the analogue of silent data corruption, for which the correct response is verification rather than checkpointing.

We specify AgentMPI, implement it in 6 900 lines of Python with all collectives built over point-to-point so that their structure is observable, and validate the implementation against closed-form cost models on 278 collective configurations with 100% agreement on message counts and fold depths. We evaluate with real agent populations on two tasks chosen to bracket the coupling spectrum: parallel translation of a book, where the dependence is weak but global, and collaborative implementation of a specified software system against a fixed acceptance suite, where it is not.

1 Introduction

In 1993 there was roughly one message-passing library per parallel machine. PVM, p4, Chameleon, PARMACS, Express, Zipcode, NX/2 and CMMD each solved the same problem differently, and a program written for one had to be rewritten for the next [92, 41, 19, 20, 74, 83]. The MPI Forum's response was not a better library but a *specification*: a portable interface that vendors could implement well and that application and library authors could target once [66, 32]. Two decades of scientific software rest on that decision.

Multi-agent systems built on language models are at the same point. AutoGen, MetaGPT, ChatDev, CAMEL, LangGraph, CrewAI and their successors each supply a coordination model, and a harness written against one cannot be moved to another or composed with another [93, 51, 78, 59, 58, 25]. The protocols that have emerged — MCP, A2A, ACP, ANP — are, on inspection, about a different problem: MCP standardises agent-to-*tool* access, and A2A standardises task delegation *across organisational boundaries* [77, 39, 2, 4]. Neither provides what an author of a multi-agent *program* needs: a way to name a subset of the population and communicate with it privately, a collective operation, a shared region with defined concurrency semantics, or a defined behaviour when a participant disappears.

The consequences are documented, and — more usefully — they are documented in terms a message-passing specification already defines. The MAST taxonomy of failed multi-agent traces attributes failures to specification and inter-agent coordination rather than to model capability [22, 13], and roughly 42% of its observed failures fall in categories that name MPI mechanisms directly: step repetition (17.1%, an idempotence property), termination-unaware and premature termination (17.6%, a barrier and agreement property), history loss and resets (5.7%, a durable log), and withheld or ignored input (1.8%, delivery semantics). We are careful about the complement: reasoning–action mismatch (14.0%) and disobeying the task specification (11.0%) are *not* protocol-addressable, and no communication layer will fix them.

Independent evidence points the same way. Mieczkowski et al. analyse language-model teams explicitly as distributed systems and find Amdahl's law bounding team speedup, decentralised teams reaching a median speedup below unity — slower than a single agent — and concurrent-write conflicts producing several times more failed tests than a coordinated arrangement [71]. Practitioners report the same outcome and attribute it to context fragmentation and information lost between agents [96, 11]. Long-context degradation and multi-turn drift are mea-

sured and substantial [61, 57, 80], and dense communication topologies are shown to *cause* premature convergence, which argues for scoped sharing as a per-operation choice rather than for more sharing [5].

These are not model problems. They are the problems a communication protocol exists to solve, and in 1993 the parallel-computing community had all of them too.

1.1 This paper

We ask what MPI actually contributes, transplant what transfers, and are explicit about what does not.

What transfers, and matters more than expected:

- **The communicator**, MPI's pair of a process group and an opaque communication context. It exists because a library and its caller could otherwise collide on an integer tag. The agent version of that collision is not hypothetical: it is a reviewer's critique reaching the wrong author, and the single-shared-conversation architecture makes it certain (Section 3).
- **Derived datatypes.** Their under-appreciated purpose is describing a *projection* of data so it can be communicated without being copied. In the agent setting this is the mechanism by which a rank receives what it needs out of a 400k-token artifact rather than the artifact (Section 4).
- **Neighbourhood collectives on a virtual topology.** An all-to-all conversation over p agents costs $\Theta(p^2)$ messages; over a ring or review graph the same information costs $\Theta(p)$ (Section 5).
- **One-sided operations with epochs and the `SEPARATE` memory model.** Every multi-agent system grows a shared scratchpad and gets its concurrency wrong the same way. MPI-3 supplies both the vocabulary and the fix (Section 6).
- **ULFM's `revoke/shrink/agree`**, whose least obvious member, `revoke`, is the one that makes recovery possible at all (Section 7).

What must change, and why the changes are the technical contribution:

C1: context is the scarce buffer, and transport mode must be visible. MPI chooses between eager and rendezvous transfer invisibly, because both deliver the same bytes. For an agent rank they do not have the same effect: an eager payload enters the agent's context window and a rendezvous envelope does not. Making the mode explicit lets us restate MPI's discipline of the *safe program* — one that does not depend on buffering — as *context safety*, and to show experimentally that the same ring-exchange program fails above a payload threshold under eager transport and completes at every size under rendezvous (Section 4, Table 7).

C2: reduction is lossy in depth, so algorithm choice is a quality decision — and the optimal tree is not binary. MPI requires reduction operators to be associative and then reorders freely; the resulting non-reproducibility of floating-point `MPI_SUM` is tolerated because it is bounded by rounding error [7, 54]. An operator implemented by a language model is not associative and is lossy, and its loss compounds with the *depth* of the reduction rather than the number of applications. We make algebraic strength part of an operator's declaration, use it to constrain algorithm selection, and report fold depth from every reduction.

Two consequences follow, and the second was not what we expected. First, a binomial tree at depth $\lceil \log_2 p \rceil$ beats a serial chain at depth $p-1$ on fidelity *and* latency — but only because broadcasts forward content-addressed handles, so relaying cannot corrupt. Second, and more interestingly: **binary is the wrong arity.** Every logarithmic collective in MPI descends from processes being *single-ported* — a process participates in one transfer at a time, so a wider tree node buys nothing. An agent rank has no port count. A prompt carries eight reports as easily as two, and merging eight in one call is one lossy re-encoding rather than seven. The binding constraint becomes the receiving rank's context budget, giving an optimal fan-in $k^* = \lfloor C(1-h)/s \rfloor$ and a fold depth of $\lceil \log_{k^*} p \rceil$. At $p = 64$, widening from k=2 to k=8 takes the fold depth from 6 to 2 *at an identical message count*, since every non-root rank sends exactly once whatever the arity. So the agent setting should prefer the *widest* feasible tree where MPI prefers the narrowest (Section 5, Table 3).

C3: the dominant failure is silent corruption, not crash. A rank that returns a confident wrong answer is invisible to timeouts, retries and schema checks. It is the analogue of silent data corruption, and HPC's answer to that is not checkpoint/restart — which preserves the corruption faithfully — but algorithm-based fault tolerance [52]. We give a six-class failure model, pair each class with the only detector that can see it, and make verification a first-class protocol concept (Section 7).

C4: protocol in the harness, not in the prompt. Because an agent's working memory is lossy and its instances are ephemeral, no protocol state may live in the agent. An agent holds only handles; every AgentMPI call is made by trusted host-side code, and the model is invoked as a kernel that transforms artifacts (Section 3).

1.2 Contributions

1. A specification of AgentMPI (Section 3–Section 8), written in the MPI standard's style with explicit rationale, covering communicators, point-to-point transfer with contracts and views, twelve collectives with declared algorithms, one-sided windows with epochs

and leases, a failure model with mitigations, and a three-axis cost model.

2. A reference implementation in which every collective is built over point-to-point, so message counts, rounds and fold depths are observable, together with closed-form cost formulas for all 27 implemented (collective, algorithm) pairs and a validation showing exact agreement on 278 configurations.

3. Two end-to-end experiments with real agent populations that bracket the coupling spectrum, plus microbenchmarks establishing the calibration constants, the context-safety threshold, fault-recovery costs and reduction fidelity.

4. A calibrated discrete-event simulator, validated against measured runs, that extrapolates collective cost to populations larger than can be run.

2 Background: what MPI contributes

2.1 The standardisation decision

MPI-1 was drafted between late 1992 and May 1994 by a forum of vendors, national laboratories and universities [65, 66]. It defined an *interface*, not an implementation, and it deliberately omitted much: no explicit shared memory, no parallel I/O, no dynamic process creation, and no fault tolerance. Those arrived in MPI-2 (one-sided operations, `MPI_Comm_spawn`, MPI-IO) [68], MPI-3 (nonblocking and neighbourhood collectives, a redesigned one-sided interface, shared-memory windows, matched probes) [69, 48, 49, 64], and MPI-4 (persistent collectives, partitioned communication, sessions) [70]. The omissions are as instructive as the inclusions: by declining to be a language, to guarantee buffering, or to handle faults, MPI-1 stayed small enough that every vendor implemented it within two years [86, 85].

2.2 The four ideas worth stealing

Communicators separate the message space from the address space. A communicator is a group plus an opaque context, and `MPI_Comm_dup` exists to manufacture a fresh context so that a library can communicate without interference from its caller [66, 43]. This is what makes parallel libraries composable, and composability is exactly what current agent frameworks lack.

Datatypes do two jobs. The first is matching: a type signature on both sides, checked at runtime, converting silent corruption into a loud error. The second, and the one that motivated the design, is describing non-contiguous data so that a boundary can be communicated without packing a copy [67, 89, 81].

Collectives are algorithms, and the algorithm matters. MPI's collective layer is built over point-to-point, and decades of work characterise which algorithm wins in which regime under the Hockney α–β model and its refinements LogP/LogGP [47, 26, 8, 88, 79, 18, 23, 82]. The results are quantitative: binomial-tree broadcast at $\lceil \log_2 p \rceil (\alpha + n\beta)$ versus flat broadcast at $(p-1)(\alpha + n\beta)$; Bruck's algorithm trading volume for rounds; Rabenseifner's reduce-scatter-plus-allgather allreduce.

Safety is a property of programs, not implementations. MPI declines to guarantee that a program depending on buffering will work, because such programs run on one implementation and deadlock on another [85]. Naming the hazard and pushing it to the programmer, with `MPI_Sendrecv` as the remedy, is a design pattern we reuse directly.

2.3 What MPI gets wrong for this setting

MPI's fault-tolerance posture is that after an error its state is undefined and the default handler aborts. FT-MPI [35] and then ULFM [17] supply mitigation primitives, but they assume fail-stop processes. MPI also has no notion of a receive buffer that cannot be resized, no notion of an operator that is only approximately associative, and no vocabulary for a participant that answers confidently and wrongly. Those gaps are what Section 4–Section 7 fill.

3 The AgentMPI model

3.1 Ranks are roles; incarnations are sessions

A job has p ranks, fixed at creation. A rank is a *durable role* defined by its number, mailbox, context budget and accounting. Its embodiment is an *incarnation*: one agent session bound to it for a bounded interval under a lease. The mapping is one-to-many over time and exactly-one at any instant.

In MPI a rank *is* a process, and the identification is harmless because MPI processes live for the whole job. Agent sessions do not: they end at a timeout, at context exhaustion, or at the host's discretion. Tying rank identity to session identity would force a communicator rebuild on every such event and invalidate the harness's mapping from rank to work — and for agents that mapping is expensive, because "rank 7 owns the parser" is baked into prompts and artifacts. Separating them makes session termination ordinary and recoverable: the rank persists, its durable mailbox persists, and a fresh incarnation resumes it. This is the virtual actor discipline [16] grafted onto MPI's static rank space.

3.2 Artifacts are immutable and content-addressed

Everything crossing a communicator is an immutable byte string addressed by the SHA-256 of its canonical serialisation. Three consequences are normative, and the first is the one the design depends on:

> *A tree broadcast forwards digests, never regenerated content.*

The obvious agent implementation of tree broadcast — "tell your children what I told you" — is a game of telephone whose corruption grows with depth, which would confine the protocol to flat, root-centred patterns and make the root a bottleneck at every scale. Forwarding handles makes a $\lceil \log_2 p \rceil$-depth broadcast deliver byte-identical content to every rank. It also draws the line that Section 5 rests on: **broadcast can be lossless because it does not transform; reduction cannot be, because it must.**

The second consequence is that admitting the same digest twice to a rank's context costs the budget once, which matters because broadcast-then-echo is pervasive. The third is that the event log plus the artifact store suffice to replay the protocol without invoking any model.

3.3 Agents hold only handles

An agent is never required to remember protocol state. It receives handles: a rank number, a communicator id, a window slot, a digest, an invocation id. We take this further than seems necessary — where an agent must issue a command, the runtime emits the *exact command string* rather than the parameters from which it could be constructed. Protocol violation in agent populations is dominated by recall failures, so a surface requiring recognition rather than recall is materially more reliable.

We can report a sharp, accidental experiment on precisely this point, because our own implementation got it wrong. In the first campaign the emitted submission command was itself malformed — an option placed after the subcommand it belonged before — and all nine workers in the population hit the parse error. All nine diagnosed it, reordered the flag, and completed their submissions, several independently reporting the defect back as something the runtime should fix. The population was robust; the mechanism was not. Two conclusions, and the second is the uncomfortable one. A generated command handed to an agent is executable code and must be tested as such, so the emitted strings are now round-tripped through the real argument parser in the test suite. And the recovery is not reassurance: these workers had shell access, could read an error, and were willing to retry. A worker that treated submission as fire-and-forget would have silently discarded work that was already correct — which is exactly the class of failure the handle discipline exists to prevent, arriving through the mechanism meant to prevent it.

3.4 Protocol in the harness, not in the prompt

AgentMPI supports two harness styles. In the *harness-side* (SPMD) style the author writes a `rank_main` executed once per rank by trusted host-side code; every AgentMPI call is made by that code and the model is invoked only through `comm.agent`. In the *agent-side* style the model itself issues protocol operations through a command-line interface.

```
def rank_main(comm):
    # decompose: variable-sized blocks, by handle
    mine = comm.scatterv(work if comm.rank == 0 else None, root=0)
    # agree a glossary: exact, idempotent, tree-parallel
    terms = comm.agent(extract_prompt(mine), contract=TERMSHEET)
    glossary = comm.allreduce(terms, ampi.UNION)
    out = comm.agent(translate_prompt(mine, glossary),
                     contract=TRANSLATION)
    # reconcile seams with immediate neighbours only
    topo = ampi.cart_create(comm, dims=[comm.size], periods=[False
        ])
    left, right = ampi.halo_exchange(topo, head(out), tail(out))
    out = comm.agent(revise_prompt(out, left, right))
    # collect by handle: the root never admits p*n tokens
    return comm.gather(out, root=0, mode=Mode.RENDEZVOUS)
```

The distinction is substantive. In the agent-side style, protocol conformance is a property of model behaviour, and a rank that forgets to enter a barrier does not merely give a worse answer — it prevents the population from making progress. Confining the protocol to host-side code makes conformance a property of the runtime. We support the agent-side style because it is sometimes the only option, and because the difference is measurable.

3.5 Protocol state lives outside the agents

MPI keeps protocol state in process memory, which is exactly as reliable as the process. For agent ranks that assumption fails twice over: the agent's working memory is a lossy context window, and the agent instance routinely disappears. So AgentMPI keeps every byte of protocol state — posted receives, unexpected messages, window contents, communicator contexts, leases — in an external store with ACID semantics, and a rank can be killed between any two calls and rebuilt from that store alone. The reference fabric is SQLite in WAL mode plus a content-addressed blob store; that is a correctness choice, not a performance one, and Section 10.1 shows the per-operation protocol cost is roughly four orders of magnitude below an agent invocation, so it is free.

4 Point-to-point and context safety

4.1 Ordering, matching, contracts

AgentMPI keeps MPI's non-overtaking guarantee: two messages from the same rank to the same rank on the same communicator that both match a receive are received in send order. The reference implementation orders by a monotone message id, giving the stronger global-FIFO

mode	payload delivered	charged to context
`EAGER`	yes	yes
`RENDEZVOUS`	envelope only	no
`SYNCHRONOUS`	yes, after match	yes
`AUTO`	by eager limit	as chosen

Table 1: Transport modes. MPI hides this choice because both modes deliver the same bytes; here they differ in whether the receiving agent's context is consumed, so the choice is semantic.

property, which is worth having because agent harnesses are debugged by reading traces.

A *contract* is a typed description with a structural part (required keys, non-empty keys, token bounds, regular expressions) and a semantic part (a natural-language post-condition carried to the consumer). Matching follows MPI: a send's and a receive's contracts match iff their names and kinds agree and the receiver's required set is a subset of the sender's; only those fields participate in matching, so a receiver may legally under-specify.

Separating *required* (key present) from *non-empty* (key present and carries a value) sounds pedantic and is not. In an early version we conflated them, and a term sheet for a book section containing no proper nouns — a correct answer — was rejected, making the rank retry, exhaust its retries, raise, and abandon peers already blocked in a collective. One over-strict contract field produced a whole-population hang.

4.2 Views: the datatype idea that matters

A *view* names a projection of an artifact without materialising it: `contiguous`, `vector(offset,count,stride)`, `indexed`, `keys`, `jsonpath`, `head`, `tail`, `digest_only`. This is MPI's second datatype job — `MPI_Type_vector` exists so a program can send a boundary column without packing a copy — and it is the job every agent framework omits. `vector` expresses a block-cyclic decomposition (give rank r every p-th chapter) with no coordinator materialising the whole. `head`/`tail` have no MPI analogue; they are the escape hatch that makes budget compliance always achievable, and because a silent truncation is a correctness hazard the runtime traces when they fire.

4.3 Transport modes and eager credit

A rendezvous envelope carries the digest, token count, contract and a one-line *synopsis*. The synopsis is not decoration: without it the receiver must fetch in order to decide whether to fetch.

Every rank publishes an *unexpected-message budget*: a bound in tokens on the volume of unmatched eager messages it accepts. A sender that would exceed the destination's budget blocks, and raises `ERR_CONTEXT_OVERFLOW` if it cannot proceed by its deadline. Both the stall and its resolution are traced.

4.4 Context safety

Definition 1. A program is *context-safe* if it completes for every assignment of unexpected-message budgets, however small.

Proposition 1. *A program in which every rank sends before any rank receives is context-safe iff its payloads travel* `RENDEZVOUS`*.*

This is a direct transplant of MPI's safe-program discipline, and the agent version is sharper. In MPI, exceeding the eager pool deadlocks or exhausts memory; here the scarce buffer is the receiving agent's context, and exceeding it silently degrades the agent's output. Making the budget explicit and blocking on it converts an invisible quality failure into a reported, attributable stall — which is the most useful thing a protocol can do about it, because the harness can then narrow a view and retry. Table 7 measures both halves.

5 Collectives and the algebra of semantic reduction

Every collective is expressed over point-to-point, so its message count, rounds and fold depth are observable. We deliberately do not short-circuit collectives through the fabric as a shared variable: the value of the abstraction is that the *shape* of the communication determines cost and quality, and a centralised implementation would hide exactly those effects. The one exception is the arrival-counting barrier, which is centralised precisely because it must be able to *name* the ranks that did not arrive.

5.1 Barriers must have a policy

An unconditional barrier is a liveness bug waiting to happen: the probability that all p agent ranks arrive within a fixed window falls with p, and "the agents are waiting for each other" is the most frequently reported multi-agent pathology [22]. Every AgentMPI barrier carries a deadline and one of five policies: `WAIT` (MPI semantics; debugging only), `RAISE`, `PROCEED` (continue with those who arrived, report absentees), `SHRINK`, `REVOKE`. The policy is the harness author's declaration of what a missing participant *means*: a missing chapter degrades a translation, a missing module kills a build.

5.2 Operators declare their algebra

An operator declares commutativity, associativity in {EXACT, APPROX, NONE}, idempotence, and an identity. The runtime enforces:

- NONE permits only the serial chain; requesting a tree is an error, not a silent quality regression.

algorithm	rounds	applications	fold depth
`chain`	$p-1$	$p-1$	$p-1$
`flat`	1	$p-1$ (all at root)	$p-1$
`binomial`	$\lceil \log_2 p \rceil$	$p-1$	$\lceil \log_2 p \rceil$
`kary`(k)	$\lceil \log_k p \rceil$	$p-1$ transfers, $\lceil \frac{p-1}{k-1} \rceil$ applications	$\lceil \log_k p \rceil$

Table 2: Reduce algorithms. The first three perform the same number of operator applications and differ only in rounds and fold depth: if quality were governed by the number of applications they would score alike, and if by depth the tree wins. `kary` additionally reduces the number of applications, because a variadic operator combines all k children in one call.

fan-in k	rounds	messages	fold depth	modelled $F(d)$
2	6	63	6	0.735
4	3	63	3	0.857
8	2	63	2	0.902
16	2	63	2	0.902

Table 3: k-ary reduction at $p = 64$, $\delta = 0.05$. The message count does not depend on k — every non-root rank sends exactly once — so widening the tree is free in volume and pays in both latency and fidelity. This is the opposite of the trade MPI faces, and it follows from agent ranks not being single-ported.

- A non-commutative operator is evaluated in rank order. A binomial tree satisfies this only when the root is rank 0; for any other root the runtime *refuses* the tree rather than silently evaluating in rotated order.
- The serial left fold is the reference semantics: for an EXACT operator every algorithm must reproduce it, and our test suite checks this across nine population sizes and every algorithm.
- Every reduction reports the *fold depth* it induced.

5.3 Fidelity decays in depth, not in width

MPI requires associativity and then reorders freely; floating-point `MPI_SUM` is consequently not bitwise reproducible, and this is tolerated because the discrepancy is bounded by rounding error [7]. An operator implemented by a language model is worse in four ways: it is not associative, it is lossy, it is expensive, and it is non-deterministic. Crucially its loss compounds with the *depth* of the reduction, because depth counts how many times an item is re-summarised on the way to the root. Modelling surviving fidelity of a d-deep fold as $F(d) = (1-\delta)^d$ for a measured per-application loss δ, Table 2 predicts that the tree dominates.

This inverts the usual expectation that parallel aggregation trades quality for speed. It holds only because of the immutability decision in Section 3: were interior ranks relaying regenerated text, a tree broadcast would incur its own depth-proportional drift, and the fidelity ordering would reverse. Table 13 measures retention.

Binary is the wrong arity. The reasoning above stops one step short. Every $\log_2 p$ factor in MPI's collective layer descends from a single hardware premise: a process is *single-ported*, so two incoming messages cost two transfers however the tree is shaped, and a wider node is pure loss. Agent ranks violate that premise completely. A prompt can carry k artifacts as easily as two, and an operator with a *variadic* kernel combines all k in one application. The constraint that replaces port count is context capacity, so the optimal fan-in is $k^* = \lfloor C(1-h)/s \rfloor$ for a budget C, per-input size s, and a reserve fraction h, and the fold depth becomes $\lceil \log_{k^*} p \rceil$.

Table 3 gives the consequence. Widening from k=2 to k=8 at $p = 64$ cuts rounds and fold depth from six to two while sending exactly the same 63 messages, because every non-root rank sends once regardless of arity. In MPI, widening a reduction tree trades latency against bandwidth; here it costs nothing and buys both latency and fidelity. AGENTMPI therefore ships `kary` with the fan-in derived from the declared context budget, and `optimal_fanin` caps it, because the relationship is not monotone in practice — a model handed fifty documents in one call attends to them unevenly, so reduced depth is eventually offset by within-call neglect. Where that cap belongs is an empirical question and the benchmark measures it.

Wide trees need variadic operators, and the runtime says so. A k-ary tree given only a binary kernel degrades to a local left fold at each node, which spends exactly the $k-1$ successive applications the wide tree was meant to avoid. AGENTMPI records which path was taken, because the failure is silent otherwise: the message count and round count look correct while the fold depth quietly reverts to $p-1$.

A third effect has no MPI counterpart. A summarising operator given a fixed output budget compresses two inputs and sixteen inputs to the same length, so the root's answer over-represents whichever subtree merged last — an unfairness `MPI_SUM` cannot exhibit, since it weights every contribution equally by construction. AGENTMPI therefore exposes the number of leaves an accumulator represents so a budget can be allocated proportionally, and reports per-rank retention so the unfairness is visible rather than inferred.

5.4 Prefer exact operators

`UNION` — set union, or key-wise union with deterministic conflict retention — is exact, associative, commutative and idempotent. A glossary merged with `UNION` is byte-identical at every rank regardless of tree shape; a glos-

sary merged by an agent is not. Harnesses should prefer exact operators wherever the combination is genuinely set-like, exactly as an MPI programmer prefers `MPI_SUM` to a user-defined operator. Idempotence additionally matters because a rank may be re-executed after a suspected but unconfirmed failure, and a non-idempotent merge applied twice double-counts.

5.5 The allreduce divergence hazard

`reduce_bcast` computes one result and broadcasts it; `recursive_doubling` has each rank compute the result itself, which is latency-optimal in MPI. The difference matters here, but not for the reason one would first assume, and the distinction is worth drawing precisely because it separates a hazard that can be engineered away from one that cannot.

The obvious worry is that under an independent-fold algorithm each rank folds in its own *arrival order*, so a non-associative operator yields p different answers. That worry is real and it is removable: ordering the operands of every pairwise combine by *rank* rather than by arrival makes both partners evaluate the identical expression tree, at no cost. It is the same reasoning MPI uses to permit trees for operators declared non-commutative, and with it a `recursive_doubling` allreduce over a deterministic operator — however lossy or order-sensitive — gives every rank the same result. We verify this, because the property is invisible to any test that uses an exact operator.

What ordering cannot fix is that a semantic operator is *nondeterministic*: every rank evaluates the same expression and gets different text. So the population still diverges under `recursive_doubling`, and **silently disagrees about the value it just agreed on** — but the cause is the operator, not the algorithm. The only remedy is for exactly one rank to evaluate and the rest to receive the result by handle, which is what `reduce_bcast` does. AgentMPI therefore defaults lossy operators to `reduce_bcast`, paying a factor of two in rounds to buy the property that agreement means agreement, and *flags* rather than forbids the alternative, since a harness whose operator happens to be deterministic should still be allowed the faster algorithm.

5.6 Scan, and topologies

Scan is the most under-used collective for agent work. A task with sequential dependence but parallel bulk — translate chapter i consistently with names established in $0..i{-}1$; write a section that must not contradict earlier ones — *is* a prefix computation. Running it sequentially costs p agent latencies; ignoring the dependence produces inconsistent output. Hillis–Steele recursive doubling delivers every prefix in $\lceil \log_2 p \rceil$ rounds at the price of $p \log p$ operator applications: it trades *more* operator work for *fewer* rounds, the opposite of the usual trade, and the right direction whenever per-invocation latency dominates (Section 8).

A virtual topology restricts who may communicate with whom, and that restriction is the main lever on the quadratic blow-up that kills multi-agent systems. A full conversation over p agents costs $\Theta(p^2)$ messages and $\Theta(p^2 n)$ tokens; over a ring or a degree-2 review graph the same information costs $\Theta(p)$ and $\Theta(pn)$. Our test suite measures the gap directly: at $p = 8$ a pairwise alltoall sends 56 messages where a degree-2 neighbourhood allgather sends 16. Halo exchange is the most valuable non-obvious transfer from HPC: a task whose parts each depend on their neighbours *looks* sequential, and expressed as a stencil it is one barrier-separated parallel step plus a constant-degree boundary exchange.

6 One-sided state: the shared blackboard

Every multi-agent system grows a shared scratchpad — a design document, a glossary, an interface file, a task board — and every one gets the concurrency wrong the same way: two agents read it, both edit a private copy, and the second write discards the first's work. MPI-3's one-sided chapter supplies the vocabulary [49, 31].

A *window* is a named, explicitly created region over a communicator, divided into *slots*; slots rather than byte offsets are the unit of locking and versioning, because the natural granularity of agent-shared state is "the section about error handling", not "bytes 4096–8191". The operations that matter are distinguished precisely:

- `put` — blind overwrite: the operation that loses work. With `expect_version` it becomes a compare-and-swap.
- `accumulate(op)` — atomic read-modify-write, so concurrent contributions combine instead of clobbering. *No lock is needed.* The operator must be exact: the read-modify-write happens inside an atomic section and an implementation cannot hold one across a model call.
- `compare_and_swap`, `fetch_and_op` — optimistic concurrency and atomic counters; the latter is how a harness implements dynamic self-scheduling, which matters more for agents than for CPUs because per-item cost varies by an order of magnitude.
- `lock`/`unlock` with a *lease*, `fence` (collective epoch boundary), `sync` (discard private copies).

`SEPARATE` is the right default. MPI-3 distinguishes a *unified* model, in which a rank always sees the latest value, from a *separate* model, in which each rank has a private copy reconciled only at synchronisation points. For an agent the private copy is not a hardware artefact: it is the copy of the document sitting in the agent's context from ten minutes ago, and it *is* stale, because peers have edited it since. `sync` is "re-read before you edit". AgentMPI

therefore defaults to `SEPARATE`, tracks each rank's observed version per slot, and records a **staleness violation** when a rank writes from a version that is no longer current.

Recording the violation is more valuable than preventing it. A harness that legitimately overwrites must still be able to, so refusing the write would break correct programs; staying silent would reproduce the lost-update bug. Our test suite asserts the *failure*: eight ranks that read, modify and blindly write a shared document produce a final value with fewer than eight contributions, and the runtime counts the stale writes that caused it. The same test with `accumulate` loses nothing and records zero stale writes.

Leases have no MPI counterpart because a dead MPI process takes the job with it. Here, a lock held by a vanished agent would deadlock the window forever, so every lock is leased and reclamation is traced — a held-then-stolen lock is exactly the situation in which two agents believe they own the same file.

Semantic updates serialise, and the trace says so. A harness needing judgement in a combine must `lock`, `get`, reason, `put` with `expect_version`, `unlock`. Making that explicit is the point: a harness that puts a semantic critical section on its critical path has built a sequential program with extra steps, and the trace shows it as lock wait time.

7 Failure model and fault tolerance

7.1 Six classes, and why F4 dominates

Table 4 gives the model. The row that matters is F4. A rank returning a confident wrong answer is undetectable by any amount of timeout tuning, retry logic or schema validation, because none of those examine whether the answer is *right*. It is the exact analogue of silent data corruption, and HPC's answer to silent data corruption is not checkpoint/restart — which preserves the corruption faithfully — but algorithm-based fault tolerance: carry redundant information that lets the computation check itself [52]. Consequently AgentMPI makes verification a first-class concept, and we argue the quantity to optimise is the *verification budget*, not the checkpoint interval that Daly's formula optimises [27].

Detection is by lease, and is *eventually perfect* rather than perfect: a slow rank is indistinguishable from a dead one. FLP says consensus is unattainable with an unreliable detector in an asynchronous system [38], and we take the same escape as HPC — assume partial synchrony with a known lease bound, accept that an expired lease *declares* a rank dead whether or not it is, and use `revoke` to make the declaration self-fulfilling so the population's view stays consistent even when the detector was wrong [24]. Straggler thresholds are derived from the run's own latency distribution, because the median-to-tail spread of an agent invocation is routinely an order of magnitude and is driven by output length rather than queueing [28]; a fixed timeout is either useless or harmful.

7.2 Revoke, shrink, agree

`revoke` makes a communicator permanently unusable for *every* member; operations already blocked inside a collective raise `ERR_REVOKED`. This is ULFM's least obvious and most necessary primitive. Without it, a rank that notices a failure cannot rescue the others, because they are all blocked waiting for the dead peer — and it cannot first reach agreement, because the group is broken. Revocation is irreversible: a communicator that could be un-revoked would let two ranks disagree about whether it is usable. Implementing this correctly required a subtlety worth recording — a blocked call must consult shared state rather than a cached flag, or it never learns it has been revoked.

`shrink` builds a communicator over the survivors, renumbered contiguously. `shrink_in_place` instead marks ranks inactive and keeps the numbering — FT-MPI's `BLANK` mode rather than its `SHRINK` mode [35] — and is often the better trade here, because renumbering invalidates a rank-to-work mapping that is expensive to recompute.

`agree` returns the conjunction over live ranks, consistently: either every survivor returns the same value or all raise. This is what makes "did the integration step succeed?" answerable when some builders died mid-step; an allreduce over logical AND would hang.

7.3 Redundancy and supervision

`replicate_and_compare` reports the modal answer, the number of distinct answers, and the consensus fraction, comparing on a harness-supplied projection because byte equality is meaningless for prose. One caution: replicating a language model does not give independent failures, since correlated errors are the norm. Agreement is evidence, not proof, which is why the consensus fraction is reported rather than a bare boolean.

MPI contributes the communication algebra and has no recovery policy; Erlang/OTP contributes the recovery discipline and no collectives [12]. The two are orthogonal — each field solved what the other ignored — and AgentMPI takes both, implementing OTP's `one_for_one`, `one_for_all` and `rest_for_one` strategies bounded by a restart intensity. The bound matters especially here: an unbounded restart loop converts a local fault into a cost overrun.

7.4 The rule harness authors break

> *A local failure must not remove a rank from a collective.*

class	description	detector	mitigation
F1 fail-stop	crashed, killed, past a hard deadline	lease expiry	re-incarnate, or shrink
F2 fail-slow	alive but past its latency budget	deadline from the run's own distribution	hedge with a duplicate
F3 fail-noisy	violates its structural contract	contract check	retry with the diagnosis
F4 fail-plausible	well-formed, confident, wrong	**independent verification only**	replicate and compare; oracle
F5 fail-greedy	exhausted its context budget	context accounting	rendezvous transport plus views
F6 adversarial	deliberate protocol deviation	—	out of scope

Table 4: The AgentMPI failure model. MPI's fault-tolerance work assumes F1; importing that assumption is why much agent-reliability engineering consists of retry loops that cannot help. F4 is the load-bearing row.

A rank that cannot compute its contribution must still enter the collective, contributing a degraded value or an identity element, and record the degradation. This is the rule most easily violated and most expensive to violate: if a local exception propagates out of `rank_main`, that rank never reaches the collective its peers are already blocked inside, and one recoverable local failure becomes a whole-population hang. We hit exactly this in our own translation harness before adding the `safe_agent` wrapper. Escalation must be a deliberate decision made by a barrier policy or a supervisor, never an accident of exception propagation.

8 Cost model

Volume is measured in tokens; a message of n tokens costs

$$T(n) = \alpha + n\beta, \qquad C(n) = n\gamma.$$

Three properties distinguish this from Hockney's model.

α is enormous, so rounds matter more than volume. For MPI, α is microseconds and bandwidth gigabytes per second, so the ratio rewards reducing message volume. For agent ranks we measure α_{p50} in the tens of seconds and effective bandwidth in the tens of tokens per second, so the ratio rewards reducing *rounds*, and latency-optimal algorithms win over a wider range of sizes than in MPI. Table 6 gives the measured constants; the crossover α/β lands at a few hundred tokens — the size of a typical agent artifact — so neither term may be neglected.

Price is an independent axis. Wall time and money are not proportional: running p ranks divides time by up to p and divides price by nothing, and coordination messages are pure added price. A harness has two objectives, and we report both everywhere.

Fidelity is a third axis. $F(d) = (1-\delta)^d$, as in Section 5.

We report Amdahl speedup, where the serial fraction f is dominated by phases that must be done by one rank — plan, integrate, arbitrate — and those do not shrink [9]; the Universal Scalability Law $S(p) = p/(1 + \sigma(p-1) + \kappa p(p-1))$, whose *coherency* term κ is positive by construction for an all-to-all conversation, so its throughput has a maximum and then declines [44]; and Karp–Flatt, which reveals a growing coordination cost that a plain speedup plot hides [56].

9 Implementation

The reference implementation is 6 900 lines of Python 3.11 with no third-party dependencies. Layers: a durable fabric (SQLite in WAL mode with `BEGIN IMMEDIATE` writers and jittered retry, plus a content-addressed blob store with fsync-and-rename); a transport with matching, non-overtaking sequence numbers and eager credit; communicators; twelve collectives with 27 algorithms; windows; fault tolerance; a cost model and calibrated simulator; and a CLI so an external agent can act as a rank.

Executors are pluggable, mirroring MPI's separation of interface from process manager: a deterministic function, a calibrated simulator, a recorded replay, and a *broker* that publishes invocations for external workers to claim. The broker is a *pull* queue, because a pushed invocation would require the harness to know how to start an agent and would couple it to a vendor; pulling lets the population be launched, scaled and re-incarnated entirely outside the harness, which is required because an agent session's lifetime is controlled by its host. The queue is per rank, because a rank is a durable role with accumulated state.

Tracing is unconditional and in the same transaction as the state change it describes. MPI's tooling interfaces are opt-in and out-of-band, which is reasonable when a run can be repeated cheaply. An agent run cannot: it is expensive and not reproducible, so a bug that was not traced cannot be investigated.

Bugs the design caught. Building the cost model and the implementation independently and cross-checking them found four real defects: a binomial scatter whose parent was computed from the highest rather than the lowest set bit (so it disagreed with the broadcast tree and deadlocked for $p \geq 4$); a chain reduce that folded from root + 1 and so broke rank order for non-commutative operators; an allreduce that reported the broadcast's fold depth instead of the reduction's; and two cost formulas

collective	algorithms	sizes	configs	message count	fold depth
allgather	3	13	31	31/31	31/31
allreduce	2	13	26	26/26	26/26
alltoall	3	13	39	39/39	39/39
barrier	2	13	26	26/26	26/26
bcast	3	13	39	39/39	39/39
gather	2	13	26	26/26	26/26
reduce	3	13	39	39/39	39/39
scan	2	13	26	26/26	26/26
scatter	2	13	26	26/26	26/26
total			278	278/278	278/278

Table 5: Measured versus closed-form message counts and fold depths, over every implemented (collective, algorithm) pair at population sizes 2–16.

parameter	value	meaning
α_{p50}	59.5 s	median agent invocation latency
α_{p99}	389.6 s	tail latency
β^{-1}	13.1 tok/s	marginal output rate
α/β	782 tok	latency/volume crossover
fabric	0.0030 s	per-operation protocol cost
α/fabric	19,840×	agent cost over protocol cost

Table 6: Cost-model constants calibrated from the translation runs. The last row is the ratio of an agent invocation to a protocol operation, and it is why keeping protocol state in a durable external store is free.

that over-counted messages at non-power-of-two sizes by up to 25%. An implementation whose measured counts disagree with its own model has a bug in one of the two, and having both is how the discrepancy gets found.

10 Evaluation

We evaluate four questions. **Q1**: does the implementation match its cost model? **Q2**: do the protocol mechanisms behave as specified under context pressure and failure? **Q3**: does an AgentMPI harness deliver a useful parallel speedup on a weakly coupled task, and do its coordination mechanisms matter? **Q4**: does it help on a strongly coupled task, where naive parallelism fails?

All agent-executed runs use a population of real agent workers claiming invocations from the broker; the worker bootstrap prompt is in the artifact and mentions neither collectives nor the experiment (Section 3). Protocol-level results (Q1, Q2) use deterministic executors so that they are free and regression-testable. Every quality number is computed by code from the agents' artifacts, never from a model's opinion of its own output; model-judged scores would make a comparison between protocol variants unreplicable.

10.1 Q1: model validation

Table 5 compares measured message counts and fold depths against the closed-form formulas. Table 6 gives the calibrated constants. The α-to-fabric ratio is the design's licence: the protocol's per-operation cost is negligible against an agent invocation, so durability, tracing and contract checking can all be unconditional.

	eager			
payload (tok)	completes	error	max ctx	comple
128	✓	–	168	✓
512	✓	–	674	✓
2,048	✓	–	2,695	✓
8,192	×	ERR_CONTEXT_OVERFLOW	0	✓
32,768	×	ERR_CONTEXT_OVERFLOW	0	✓

Table 7: Ring exchange in which every rank sends before any rank receives, at $p = 8$ with an unexpected-message budget of 4,096 tokens. Eager transport completes only while the payload fits the budget; rendezvous completes at every size.

policy	survivors complete	detect (s)	absentees named	shrink (s)	agreer
`wait`	×	15.25	×	–	
`raise`	×	15.25	×	–	
`proceed`	✓	15.20	✓	–	
`shrink`	✓	15.20	✓	0.000	

Table 8: One rank of eight fails silently before a barrier. `WAIT` is MPI's only option and never completes.

10.2 Q2: context safety and failure recovery

Table 7 confirms the context-safety proposition with a sharp threshold, and shows the failure is *reported* as `ERR_CONTEXT_OVERFLOW` rather than manifesting as a hang or a silent truncation. Table 8 shows that `WAIT` and `RAISE` do not let the survivors complete, while `PROCEED` and `SHRINK` do, correctly name the absentee, and reach consistent agreement.

10.3 Q3: parallel book translation

Translating a book looks embarrassingly parallel and is not: sections disagree about proper nouns, drift in register, and show visible seams. The dependence is weak but real, and has the shape of a stencil — mostly local work, a little that must be globally consistent, a little that must agree with immediate neighbours. Our harness is the corresponding HPC decomposition (Section 3): `scatterv`, agent-local term extraction, `allreduce` with `UNION` to agree a glossary in $2\lceil \log_2 p \rceil$ rounds with no coordinator reading any section, translation, `halo_exchange` on a 1-D Cartesian topology to reconcile seams, and `gather` by handle so the root never admits pn tokens.

The protocol delivers a real speedup with no coherency ceiling. Table 10 shows 3.46× speedup at $p = 8$ on identical total work, at 43% efficiency. The interesting number is the USL fit, $\sigma = 0.220$, $\kappa = 0.0000$ ($R^2 = 0.873$): the *contention* coefficient σ is substantial at 0.22, and the *coherency* coefficient κ is zero. That

configuration	consist.	diverg.	verified	para.	CJK	wall (s)	calls	tokens	USD
glossary, halo	1.000	0/6	1.000	1.000	0.858	167	24	52,084	0.260
glossary, halo, rec. doubling	1.000	0/6	1.000	1.000	0.859	158	24	52,458	0.267
glossary, halo, **semantic merge**	1.000	0/6	1.000	1.000	0.859	1,153	31	62,408	0.354
no glossary, halo	1.000	0/6	1.000	1.000	0.857	95	16	20,108	0.136
glossary, **no halo**	1.000	0/6	1.000	1.000	0.858	126	16	34,054	0.163

Table 9: Translation ablations at the largest population, with real agent ranks. "consist." is the fraction of entities rendered identically by every section that rendered them; "verified" is the fraction of reported renderings that actually occur in the reporting section's text, which is a direct measurement of fail-plausible behaviour.

p	wall (s)	speedup	efficiency	Karp–Flatt	calls	tokens	USD	consist.
1	577	1.00	1.00	0.000	16	33,756	0.161	1.000
2	410	1.41	0.70	0.422	18	38,253	0.186	1.000
4	308	1.87	0.47	0.380	22	46,926	0.231	1.000
8	167	3.46	0.43	0.188	24	52,084	0.260	1.000

Table 10: Strong scaling: identical total work, decreasing population. USL fit: $\sigma = 0.220$, $\kappa = 0.0000$ ($R^2 = 0.873$).

is the distinction the model exists to draw. A positive κ would mean the coordination cost grows superlinearly and throughput has a maximum beyond which adding agents makes the system slower — the signature of an all-to-all conversation, and the ceiling that group-chat architectures hit. Measuring $\kappa = 0$ says this harness's cost is a constant serial fraction, not a ceiling, which is precisely what restricting communication to collectives and a bounded-degree topology is supposed to buy. The Karp–Flatt metric agrees: it does not grow with p.

The coordination machinery has a measured price. Reading Table 9 as a cost table rather than a quality table: relative to translating with neither mechanism, adding the glossary `allreduce` and the halo exchange costs 76% more wall time and 159% more tokens. That is what agreement costs, quantified, and it is the number a harness author needs before deciding to pay it.

Exact operators are worth reaching for, and the margin is large. The same glossary agreement performed by a semantic `allreduce` — an LLM-implemented merge — rather than by the exact, idempotent `UNION` cost 6.9× the wall time, 1.2× the tokens and 1.4× the price, and produced glossaries of the same size with identical measured quality. It also raised the agent-call count from 24 to 31, because a semantic reduction adds one model invocation per tree node on top of the per-section work. This is the strongest practical result in the experiment and it is a direct analogue of preferring `MPI_SUM` to a user-defined operator: when a combination is genuinely set-like, an exact operator is not merely cheaper but exactly reproducible, whereas the semantic version is neither. Choosing `recursive_doubling` over `reduce_bcast` for the exact operator, by contrast, changed almost nothing (158 s versus 167 s), which is what the cost model predicts when p is small enough that a factor of two in rounds is within run-to-run variance.

An honest negative result. On this text, they did not have to. All configurations achieved perfect entity consistency, including the one with no glossary exchange at all, because the pre-registered entities are famous enough that competent models render them identically without coordination. We report this rather than substituting a metric after seeing the data. Two things follow, and both are useful. First, our consistency metric is saturated on canonical entities and a sensitive replication must score rare names (Section 11). Second, and more generally: *a coordination mechanism only pays where the participants would otherwise disagree*, so a harness should spend agreement on the terms that are genuinely ambiguous and not on the ones everybody already knows. The protocol makes that a decision the author can price, which is more than a framework that broadcasts everything to everyone can offer.

Two quality checks did not saturate and are worth recording. Every reported entity rendering actually occurred in the text that reported it, so on this task we observed no fail-plausible behaviour on the one axis where we could mechanically detect it. And paragraph structure was preserved exactly in every section, which is the contract the harness enforced and which a free-running translation agent routinely violates.

10.4 Q4: collaborative software development

The complement to translation. Eight agents each handed one module of a specification produce eight modules that do not compose, because each invented its neighbours' interfaces. The harness is organised around exactly that failure: `bcast` the specification; publish interfaces into a window slot per module and `fence`; read dependencies' interfaces in a fresh access epoch; implement; `barrier`; run the acceptance suite at rank 0; `scatter` per-module blame (not broadcast everything — a rank should receive its own failures); `neighbor_allgather` for degree-2 cross review; renegotiate an interface under an exclusive lock; `agree` on whether the build is green.

The oracle is a 59-case acceptance suite written before the run and never shown to the agents as a target to optimise, run out of process because generated code can

configuration	p	passed	lines	defensive	/kloc	wall (s)	calls	tokens	USD
shared interfaces	8	60/60	1,954	7	3.58	499	32	161,989	1.145
no shared interfaces	8	59/60	2,693	33	12.25	529	32	167,174	1.377
single agent (p=1)	1	60/60	1,752	1	0.57	1,213	2	31,945	0.438

Table 11: Collaborative implementation of the `minidb` SQL engine against a fixed 59-case acceptance suite written before the run. "per round" is the number of cases passing after each integration round.

configuration	locks	contended	wait (s)	max wait	puts	waits	mean wait	retries
single agent (p=1)	0	0	0	0.00	0	0	0.000	0
shared interfaces	8	0	0.00	0.00	16	0	0.000	0

Table 12: Contention on the shared interface window.

hang or exhaust memory. It is also our model of verification (Section 7): each case is an independent check that a confident output actually behaves as specified.

Table 11 reports pass rates and Table 12 the window contention, which bounds the achievable speedup Amdahl-style.

The parallel population buys latency with money, at equal quality. The headline comparison is against the baseline a multi-agent system has to beat. Eight agents reached 60/60 in 499 s using 32 agent invocations; one agent writing all eight modules itself reached 60/60 in 1213 s using 2. That is a 2.43× latency reduction at 30% parallel efficiency, bought for 2.6× the price and 5.1× the tokens.

We think this is the right way to report a multi-agent result, and it is not the way they are usually reported. The population did not achieve anything the single agent could not; the task is specified precisely enough that one agent gets there in two passes. What the population bought was wall-clock time, and what it paid was money and tokens — which is exactly what Section 8's insistence on price as an *independent* axis predicts, since running p ranks divides time by up to p and divides price by nothing. A harness author facing this trade needs both numbers, and a paper that reported only the speedup would be concealing the more important one. It also explains, without any appeal to model capability, the widely reported finding that multi-agent architectures frequently fail to beat a single agent [96, 71]: on a well-specified task with no latency pressure, there is nothing for them to win.

The single agent could verify continuously; no rank could. The baseline's result comes with an asymmetry we should have anticipated and did not, and it is structural rather than an artefact. The single agent, asked for all eight modules at once, wrote them to a scratch directory, wrote itself a 49-case test suite covering the specification's NULL logic, aggregate rules, ordering and error-wrapping guarantees, and iterated until they passed — *before* submitting anything. It reported doing so unprompted.

No rank in the parallel population could do that. A rank owns one module of eight and cannot execute the system, so for it the first opportunity to observe whether anything works is the integration barrier. Decomposition does not merely divide the work; it divides the work into pieces that are individually *untestable*, and it moves verification from something continuously available to something available once per round. That is a real disadvantage of parallel agent development, it is invisible in a pass-rate table, and it is not fixed by better prompting.

It does suggest a concrete protocol-level remedy, which we did not implement and would in a revision: verification should be reachable *inside* a rank's loop rather than only at the barrier. A rank could pull the current contents of the shared window, assemble the system from its peers' latest published artifacts, and run the acceptance suite locally — speculatively, against possibly-stale peers, in the spirit of the `SEPARATE` memory model where a rank knowingly reasons about a private copy. The protocol already has the pieces; the harness simply did not use them, and Section 7's claim that verification is the load-bearing fault-tolerance mechanism implies it should have.

Eight agents composed a working system on the first integration. With interfaces published into the RMA window, the population produced roughly 1 950 lines across eight modules — tokeniser, AST, parser, planner, expression and aggregate functions, execution engine, public API — and passed the entire acceptance suite at the first integration barrier. No agent saw another agent's code at any point; each was given the specification, its own module's assignment, and the published interfaces of the modules it depends on. That a parser written against a declared `tokenize` signature composes with a planner written against a declared AST, without either author seeing the other's implementation, is the whole claim of the interface-publication phase, and it is the outcome an ordinary "eight agents, eight modules" prompt does not produce.

Withholding interfaces does not cost correctness; it costs defensive coupling. This is the experiment's most interesting result and we did not anticipate it. Removing the shared-interface window cost exactly one acceptance case out of sixty — a near-null result on the metric we had chosen. But the agents told us why in their own reports: deprived of their dependencies' published signatures, they did not fail, they *adapted*. One re-classified tokens by value because it could not know the token-kind vocabulary; another constructed AST nodes

by matching values onto whatever field names the class actually declared; a third reached for `getattr` fallbacks throughout. Each independently, and each without being asked to.

That behaviour is invisible to an acceptance suite and plain in the source, so we measured it. Counting, with the AST rather than by pattern match, the constructs an author writes when guessing at a collaborator's shape — three-argument `getattr`, `hasattr`, and handlers for the exceptions a wrong guess raises — gives 3.58 per thousand lines with the window and 12.25 without it, a factor of 3.4, alongside 38% more code (1,954 lines versus 2,693). The single agent, which owned every module and therefore never had to guess, wrote 0.57 per thousand lines.

So the price of missing interface information is paid in runtime introspection rather than in failed tests: the modules still compose, by each one defensively accommodating whatever its neighbours turned out to be. That is a real cost and a familiar one — it is how human codebases decay when interfaces are informal — and it is exactly the cost a pass-rate metric cannot see. We offer *defensive constructs per thousand lines* as a cheap, deterministic, and oracle-independent measure for multi-agent coding work generally; it needs no judge and no test suite, only the source.

The pass-rate ablation was also confounded, and the confound is instructive. Beyond the adaptation above, there is a second reason the pass rate barely moved. The cause is a flaw in our experimental design: the broadcast specification already lists each module's exported signatures, so *the specification is itself a shared interface*, and the window had nothing left to contribute. A rank could read `tokenize(sql) -> list[Token]` out of the spec whether or not its author had published anything.

The lesson is worth more than the ablation would have been: **interface publication pays only to the extent that the specification underdetermines the boundaries.** A precisely specified system does not need a negotiation phase, and paying for one is waste; an underspecified system — which is the normal case, and the case in which eight agents each invent their neighbours' signatures — does. This also gives the mechanism a decision rule rather than a recommendation: measure how much of your interface surface the specification pins, and publish only the rest.

We therefore added a *vague-specification* variant in which the module table names responsibilities and import permissions but no signatures at all, leaving six of the eight module boundaries to be negotiated through the window. That pair — vague specification with and without the window — is the experiment that actually tests the mechanism, and it is the comparison we should have designed first.

A directed collective on the wrong graph made peer review a no-op. An agent rank reported that the critiques it received named only a file it did not own. It was right, and the cause is a mistake worth naming because the protocol makes it easy to make. `review_edges` yields *(author, reviewer)* pairs, so a rank's graph *sources* are the authors whose code it reviews. Returning the critique with a neighbourhood collective over that same graph sends it to the rank's *destinations* — the ranks it sends its own code to — which is a disjoint set. Every author therefore received critiques of other people's modules, and the review phase produced text that reached nobody who could act on it.

The fix is to return reviews over the *transposed* graph, which is two lines. The general lesson is that a neighbourhood collective on a directed topology has a direction, and a request/response pair needs *both* the graph and its transpose — a distinction MPI's `MPI_Dist_graph_create` makes available (it takes sources and destinations separately) and which we simply failed to use. Notably, nothing in the run looked wrong: message counts, degrees and round counts were all correct, because the traffic was well-formed and merely addressed to the wrong ranks. It was found by an agent noticing the semantics, not by any test; there is now a test.

This compounds the confound above. In the precise-specification runs, the review phase was inert *and* the interface window was redundant, so two of the three coordination mechanisms under study were doing nothing — which is a further reason to read that ablation as uninformative rather than negative. The vague-specification pair runs with corrected routing.

Two further defects in our own measurement, both worth reporting. The first was in the oracle. One acceptance case tested case-insensitive *keywords* using a case-varied *identifier*, which the specification says is case-sensitive; the population implemented the specification correctly and the suite marked it wrong. An oracle is a program and can be the defective party, which matters directly for Section 7's argument: a fault-tolerance scheme built on verification inherits the reliability of its verifier, and we should not have claimed otherwise without saying so.

The second was worse and less visible. The harness parsed the suite's JSON report by slicing from the *last* brace in its output — which lands inside a nested per-case object and never parses — so every run was recorded as failing to import with zero passes. Because that report is precisely what the harness scatters back to the population as the definition of done, **the agents were told a build that passed 58 of 59 cases had failed to import**, and spent their repair round on a phantom. A bug in the plumbing *around* an oracle is as damaging as a bug in the oracle itself, and neither is visible from the population's behaviour.

We report the consequences rather than hiding them. Every configuration received the same false signal, so the ablation comparison is between equally handicapped

runs and remains valid; the pass rates in Table 11 come from re-scoring the agents' code offline against a single corrected oracle (`scripts/reeval_software.py`), which is free because the code is on disk. What we cannot claim from this campaign is anything about *repair* dynamics: the second round was driven by a report that was not true, so the interesting question of whether a population converges under honest failure feedback is untested here. Notably, the population reached a full pass rate anyway — it was robust to being told, falsely, that its correct work had failed.

10.5 Reduction fidelity

Table 13 tests the central prediction of Section 5: since all algorithms perform the same number of operator applications, retention differences must be attributable to fold depth.

The depth penalty is conditional, and the condition is capacity. Our first configuration — eight ranks contributing four identified items each, merged under a 900-token budget — produced *perfect* retention for all three binary algorithms, with zero fabricated identifiers, whether the fold depth was 3 or 7. The reason is straightforward once seen: thirty-two short items fit the output budget, so the operator was never forced to discard anything, and an operator that discards nothing is lossless at any depth.

This is a more useful claim than the one we set out to test. **Fold depth penalises fidelity only when the reduction is capacity-bound**, so the first question a harness author should ask is not which tree to use but whether their reduction is saturated. If it is not, the shallowest tree is free and the fidelity argument is moot; if it is, depth is the quantity to minimise. Table 13 therefore reports a deliberately saturating configuration alongside the unsaturated one.

At equal quality, the tree wins outright on latency. The unsaturated configuration is still informative, because it isolates the cost axis with quality held fixed at 1.0: the binomial tree completed in 58 s where the serial chain took 131 s and the flat reduction 153 s, all performing exactly seven operator applications — a 2.2× latency reduction for free. That flat is the *slowest* despite needing one communication round is the cost model's prediction confirmed: all seven applications execute serially at the root, so its critical path is 7α rather than $\lceil \log_2 8 \rceil \alpha$. The manager-summarises-everything pattern is the worst of the three on wall time and has no compensating advantage on quality.

10.6 Simulated scaling

No agent-systems paper can afford a 1024-rank experiment, and none should pretend otherwise. HPC's honest answer is to build a discrete-event simulator of the communication pattern, calibrate it against a real run at an affordable size, and extrapolate [50]. Table 14 is that extrapolation, and Table 15 the resulting algorithm-selection crossovers — the classical form of a collective-tuning result, now with a fidelity column that has no MPI counterpart.

11 Discussion and limitations

Scale. Our real-agent populations are single-digit, because the concurrency limit of our agent host bound them, and because the campaign structure — one persistent worker pool serving a whole ablation ladder — trades population size for the ability to run many configurations against the same agents. Larger populations are reported only as calibrated simulation and labelled as such. The protocol-level results are size-independent and validated to $p = 32$.

The insensitive metric. As noted in Q3, our entity-consistency metric saturates on famous proper nouns. A more sensitive replication should ask each rank to report renderings for *every* proper noun it used rather than a pre-registered list, and score consistency over terms reported by two or more ranks; rare names diverge where famous ones do not. We report the saturated result rather than substituting the metric after seeing the data.

An exact operator can impose a locally wrong global decision, and our metric rewards it. A worker rank surfaced a case we had not anticipated. The `UNION` glossary bound the source term *pounds* to the rendering for pounds sterling, correctly for most of the book; in one section the word denotes body weight, and the rank — correctly following a glossary the harness declared binding — produced a semantically wrong sentence. The section that received no glossary rendered it naturally.

This is a real limitation of the design, not an implementation slip. An exact, order-independent merge over a term map presupposes that a source term has one correct target rendering, which polysemy falsifies; `UNION`'s conflict retention handles two ranks *disagreeing* about a term but not one term having two correct senses. The fix is a contract change — key the glossary on (term, sense) and let a rank declare which sense it saw — and it makes the shared state larger and the agreement weaker, which is exactly the trade the design should have surfaced.

Worse for our evaluation: **the consistency metric scores this failure as a success**, because it counts identical renderings across sections and a globally imposed wrong rendering is maximally consistent. A metric that rewards agreement cannot detect agreement on something false. That is a specific instance of the general hazard in Section 7 — verification inherits the reliability of its verifier — and it means our perfect consistency scores should be read as evidence about *coordination*, which is

algorithm	depth	rounds	retention	sd	rank corr.	halluc.	wall (s)	out tok
agent-executed: p=8, 4 items/rank — **not capacity-bound**								
`chain`	7	7	1.000	0.000	0.000	0	131	3,027
`flat`	7	1	1.000	0.000	0.000	0	153	3,033
`binomial`	3	3	1.000	0.000	0.000	0	58	2,180
surrogate operator (function): p=16, 4 items/rank — **capacity-bound**								
`chain`	15	15	0.688	0.464	-0.775	0	1	241,979
`flat`	15	1	0.875	0.331	0.164	0	1	2,323,398
`binomial`	4	4	0.703	0.444	0.347	0	0	11,833
`kary`, k=4	2	2	0.688	0.464	0.044	0	0	3,723

Table 13: Fact retention of a semantic reduction. Each rank contributes a report of uniquely identified factual items; retention is counted mechanically at the root. "rank corr." is the correlation between a rank's index and the fraction of its items that survived — a positional unfairness `MPI_SUM` cannot exhibit.

collective	algorithm	p=8	p=32	p=128	p=512	p=8192
allgather	`bruck`	0	0	0	0	0
allgather	`ring`	0	0	0	2	3
allreduce	`recursive_doubling`	159	290	451	596	682
allreduce	`reduce_bcast`	159	290	451	596	682
alltoall	`bruck`	0	0	0	0	0
alltoall	`pairwise`	0	0	0	2	3
barrier	`dissemination`	0	0	0	0	0
barrier	`linear`	0	0	0	2	3
bcast	`binomial`	0	0	0	0	0
bcast	`chain`	0	0	0	2	3
bcast	`flat`	0	0	0	2	3
reduce	`binomial`	159	290	451	596	682
reduce	`chain`	329	1,416	6,104	24,246	48,737
reduce	`flat`	329	1,416	6,104	24,244	48,734

Table 14: Simulated collective makespan (seconds) with parameters calibrated from the measured runs, median of 32 trials. Latency is drawn log-normally, because agent latency is right-skewed and a collective's completion time is a maximum over ranks.

what they measure, and not about translation quality, which they do not.

We committed the lost-update bug in the harness that runs the experiments. Two campaigns sharing a campaign directory ran concurrently; the second activated its job, then the first finished and cleared the shared pointer with a blind write, stranding the second's worker pool with nothing to poll. A reduction sat waiting on a rank that could no longer find its work. That is exactly the two-writers-no- synchronisation failure that Section 6 is about, and the remedy is the one that section prescribes — clear the cell only if it still holds your value, a compare-and-swap. We record it because it is the paper's own argument arriving as a self-inflicted wound: the discipline is easy to state, easy to skip, and its violation fails silently rather than loudly. A protocol that makes the safe operation available does not make anyone use it, which is an argument for defaults, not for documentation.

Shared mutable code is shared state the protocol does not cover. One worker crashed with a type error inside the runtime, and the cause was that we edited the package — installed in editable mode — while a live population was executing against it, so that worker observed a half-completed refactor. The protocol keeps protocol *state* outside the agents and durable, which is the design's central move, and it does nothing at all about the *code* being shared and mutable. A production deployment should pin an immutable runtime version per job and have workers report a mismatch; ours did not, and the honest description of the failure is that we hot-patched a running job.

One model family. All agent ranks were served by one model family, so we cannot separate protocol effects from model idiosyncrasies, and the correlated-failure caution in Section 7 applies to our own replication results.

Token counting. Where provider usage was unavailable we used a documented estimator; every conclusion depending on exact counts is drawn from ratios rather than absolute values.

Fabric as a fate-sharing point. The reference fabric is a single durable store, so `agree` routes agreement through a reliable service rather than circumventing FLP. The specification states that requirement rather than hiding it; a distributed fabric would need real consensus.

What we would change. The context budget is currently a scalar. Real agents have structured contexts — system prompt, tool definitions, conversation, retrieved documents — with very different eviction economics, and a protocol that modelled the structure could make better admission decisions.

12 Related work

MPI and its lineage. We build directly on the MPI standard and its algorithmic literature [66, 69, 70, 88, 23, 18, 79], its one-sided design [49, 31, 42], and its fault-tolerance line [35, 17]. Our departures are catalogued in Section 1.

Other parallel models. BSP's superstep discipline [91] is the ancestor of our barrier-separated phases; Pregel

collective	tokens	from p	best (time)	best (volume)	best (energy)
bcast	64	2	`binomial`	`binomial`	`binomial`
bcast	512	2	`binomial`	`binomial`	`binomial`
bcast	4,096	2	`binomial`	`binomial`	`binomial`
bcast	32,768	2	`binomial`	`binomial`	`binomial`
reduce	64	2	`binomial`	`binomial`	`binomial`
reduce	512	2	`binomial`	`binomial`	`binomial`
reduce	4,096	2	`binomial`	`binomial`	`binomial`
reduce	32,768	2	`binomial`	`binomial`	`binomial`
reduce	64	4	`flat`	`binomial`	`binomial`
reduce	512	4	`flat`	`binomial`	`binomial`
reduce	4,096	4	`flat`	`binomial`	`binomial`
reduce	32,768	4	`flat`	`binomial`	`binomial`
allreduce	64	2	`recursive_doubling`	`recursive_doubling`	`recursive_doubling`
allreduce	512	2	`recursive_doubling`	`recursive_doubling`	`recursive_doubling`
allreduce	4,096	2	`recursive_doubling`	`recursive_doubling`	`recursive_doubling`
allreduce	32,768	2	`recursive_doubling`	`recursive_doubling`	`recursive_doubling`
allgather	64	2	`bruck`	`bruck`	`bruck`
allgather	512	2	`bruck`	`bruck`	`bruck`
allgather	4,096	2	`bruck`	`bruck`	`bruck`
allgather	32,768	2	`bruck`	`bruck`	`bruck`
alltoall	64	2	`bruck`	`bruck`	`bruck`
alltoall	512	2	`bruck`	`bruck`	`bruck`
alltoall	4,096	2	`bruck`	`bruck`	`bruck`
alltoall	32,768	2	`bruck`	`bruck`	`bruck`
alltoall	64	4	`linear`	`linear`	`linear`
alltoall	512	4	`linear`	`linear`	`linear`
alltoall	4,096	4	`linear`	`linear`	`linear`
alltoall	32,768	4	`linear`	`linear`	`linear`
barrier	64	2	`dissemination`	`central`	`central`
barrier	512	2	`dissemination`	`central`	`central`
barrier	4,096	2	`dissemination`	`central`	`central`
barrier	32,768	2	`dissemination`	`central`	`central`
barrier	64	4	`central`	`central`	`central`
barrier	512	4	`central`	`central`	`central`
barrier	4,096	4	`central`	`central`	`central`
barrier	32,768	4	`central`	`central`	`central`

Table 15: Algorithm recommendations under three objectives. Where the time-optimal and fidelity-optimal choices differ, the harness author has a decision to make that MPI never presents.

applies it to graphs [62]. The actor model [45, 6] and Erlang/OTP [12] supply supervision, which MPI lacks and which we adopt wholesale; Orleans supplies the virtual-actor idea behind rank incarnations [16]. CSP contributes the rendezvous channel [46]. MapReduce and Spark show how restricting expressiveness buys recoverability [29, 98]; we make the opposite trade and pay for it with an explicit failure model. Task-graph runtimes [14, 72] and pipeline parallelism [53] inform our topology work.

Agent frameworks. AutoGen, MetaGPT, ChatDev, CAMEL, LangGraph and CrewAI each supply a coordination model, and each is a framework rather than a protocol: they own the control flow, and a harness written against one cannot be composed with another [93, 51, 78, 59, 58, 25]. None provides scoped communication contexts, collectives with selectable algorithms, one-sided state with defined concurrency semantics, or a specified behaviour when a participant disappears. Durable-execution systems [87] give reliable step execution but no communication algebra.

Agent protocols. MCP standardises agent-to-tool access [77]; A2A standardises task delegation across organisations [39]; ANP, ACP and Coral address discovery and routing [2, 4]. Surveys confirm the division [97, 33]. These are complementary and orthogonal: they connect *systems*, whereas AgentMPI is what one writes a system *with*. Two details make the orthogonality sharper than it first appears. MCP's current revision moved deliberately *away* from shared state, making requests stateless and self-contained with per-request capability negotiation, specifically to avoid stateful load balancing [77, 76]; it is a tool protocol hardening into statelessness, not drifting toward agent-to-agent. And A2A's `contextId` is a correlation identifier rather than a membership set: its unit of coordination is the bilateral task, with retry and circuit-breaking explicitly left to the client [39, 75]. Neither has a group, and without a group there is no collective.

Classical agent communication languages — KQML, FIPA-ACL, the Contract Net Protocol, blackboard architectures, Linda tuple spaces [36, 37, 84, 34, 21, 15] — are the true ancestors, and modern protocols are rediscovering them, notably, Linda's tuple space is a shared-memory abstraction with no epochs, which is precisely the gap Section 6.1 fills.

The closest prior work, and the objection it raises. Mieczkowski et al. [71] is simultaneously our strongest motivation and the paper a reviewer will ask about. It validates the distributed-systems analogy empirically — Amdahl bounds, sub-unity speedup for decentralised teams, concurrent-write conflicts as a dominant failure — and it is the work we would cite to justify the whole enterprise. It produces analysis and design guidance, not an interface. Our claim is therefore narrower and more specific than "agent teams are distributed systems": it is that a particular thirty-year-old interface transfers, that three named modifications are forced, and that the modifications are measurable. We are also aware of at least one independent design sketch proposing MPI-shaped primitives for agents (communicators, deterministic reduce, one-sided put/get, shrink and agree) in a project issue tracker rather than a publication; we claim priority on none of the vocabulary, only on the specification, the implementation, and the measurements.

Reduction cost in MPI's own literature. Two results deserve particular credit because they anticipate our concerns from inside MPI. The commutativity flag on `MPI_Op_create` is already an algorithm-selection gate rather than documentation, which is exactly the mechanism we generalise to a three-valued associativity declaration. And Träff's recent scan work is explicitly motivated by operators whose *application* is expensive, minimising the number of operator applications subject to a bound on rounds [90] — the closest thing in the MPI literature to an algorithm designed for an LLM cost model, and the same objective our `kary` reduction optimises. On reproducibility, the contrast between Intel's conditional numerical reproducibility (reproducible only at a fixed

thread count) and ReproBLAS's binned accumulator (reproducible regardless of order or process count) [54, 30, 7] makes the design argument for us: fix the *operator*, not the order — which is why Section 5 pushes harnesses toward exact, order-independent operators like `UNION` wherever the combination is genuinely set-like.

Agent failure and context. The MAST taxonomy attributes multi-agent failures largely to specification and inter-agent coordination [22, 13]; practitioner reports converge on context fragmentation [96, 11, 10, 40]. Long-context degradation is measured [61, 57, 80], as are error cascades and diversity collapse [5]. Agent-OS work treats context as a managed resource [73, 63]; serving systems schedule agent workloads [60, 3, 1]. Our contribution relative to all of these is to make context an *accounted protocol resource* with a transport-mode policy, rather than a memory-management strategy inside one agent.

Multi-agent translation and coding. Multi-agent literary translation has been studied [95, 94], and coding benchmarks are standard [55]. We use both as *vehicles* for measuring protocol mechanisms, not as targets; our harnesses are deliberately simple so that ablations isolate the coordination primitive rather than prompt engineering.

13 Conclusion

The multi-agent field is rediscovering, application by application, the problems the MPI Forum solved once for parallel computing. We have shown that MPI's core abstractions — communicators, typed transfers with projections, collectives with selectable algorithms, one-sided state with epochs, and failure mitigation — transfer to agent populations, and that three modifications are forced and consequential: transport mode must be semantically visible because context is the scarce buffer; reduction operators must declare their algebra because fidelity decays in fold depth; and fault tolerance must be built around verification because the dominant failure is silent corruption rather than crash. What makes the transplant work is one decision that MPI never had to make and that agent frameworks never make: artifacts are immutable and content-addressed, which is why broadcast can be logarithmic and lossless while reduction cannot.

We do not claim AgentMPI's spelling is right. MPI's was not right either in 1994, and the value of the Forum was the process, not the first draft. We claim that the *shape* of the problem is the same one message passing already solved, and that a field currently shipping one coordination mechanism per project should look at what happened after 1994.

References

[1] Teola: Towards end-to-end optimization of llm-based applications. arXiv:2407.00326, 2024, 2024.

[2] Agent network protocol technical white paper. arXiv:2508.00007, 2025, 2025.

[3] Autellix: An efficient serving engine for llm agents as general programs. arXiv:2502.13965, 2025, 2025.

[4] Coral protocol: Open infrastructure connecting the internet of agents. arXiv:2505.00749, 2025, 2025.

[5] Diversity collapse in multi-agent llm systems: Structural coupling and collective failure in open-ended idea generation. Findings of ACL 2026, pp. 13. DOI 10.18653/v1/2026.findings-acl.13 ; arXiv:2604.18005. Code:, 2026.

[6] Gul Agha. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, 1986.

[7] Peter Ahrens, Hong Diep Nguyen, and James Demmel. Efficient reproducible floating point summation and blas. Technical Report UCB/EECS-2015-229, EECS Department, University of California, Berkeley, 2015.

[8] Albert Alexandrov, Mihai F. Ionescu, Klaus E. Schauser, and Chris Scheiman. Loggp: Incorporating long messages into the logp model for parallel computation. *Journal of Parallel and Distributed Computing*, 44(1):71–79, 1997.

[9] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proc. AFIPS Spring Joint Computer Conference*, pages 483–485, 1967.

[10] Anthropic. Effective context engineering for ai agents. Engineering blog.

[11] Anthropic. How we built our multi-agent research system. Engineering blog, 13 June 2025, 2025.

[12] Joe Armstrong. *Making Reliable Distributed Systems in the Presence of Software Errors*. PhD thesis, Royal Institute of Technology (KTH), Stockholm, 2003.

[13] Same authors. Why do multi-agent llm systems fail? NeurIPS 2025 poster #121528, 2025.

[14] Michael Bauer, Sean Treichler, Elliott Slaughter, and Alex Aiken. Legion: Expressing locality and independence with logical regions. In *Proc. International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2012.

[15] Bellifemine, F., Poggi, A., Rimassa, G. Jade — a fipa-compliant agent framework. PAAM 1999, 1999.

[16] Philip A. Bernstein, Sergey Bykov, Alan Geller, Gabriel Kliot, and Jorgen Thelin. Orleans: Distributed virtual actors for programmability and scalability. In *Microsoft Research Technical Report MSR-TR-2014-41*, 2014.

[17] Aurélien Bouteiller, George Bosilca, Guillaume Mercier, and Thomas Hérault. Implicit actions and non-blocking failure recovery with MPI. arXiv:2212.08755, 2022. ULFM design requirements; comparison against FT-MPI, MPI Reinit, FMI; "a flexible low-level API that supports a variety of fault tolerance models".

[18] Jehoshua Bruck, Ching-Tien Ho, Shlomo Kipnis, Eli Upfal, and Derrick Weathersby. Efficient algorithms for all-to-all communications in multiport message-passing systems. *IEEE Transactions on Parallel and Distributed Systems*, 8(11):1143–1156, 1997.

[19] Ralph Butler and Ewing Lusk. Monitors, messages, and clusters: The p4 parallel programming system. *Parallel Computing*, 20(4):547–564, 1994.

[20] R. Calkin, Rolf Hempel, Hans-Christian Hoppe, and P. Wypior. Portable programming with the PARMACS message-passing library. *Parallel Computing*, 20(4):615–632, 1994.

[21] Nicholas Carriero and David Gelernter. Linda in context. *Communications of the ACM*, 32(4):444–458, 1989.

[22] Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., Stoica, I. Why do multi-agent llm systems fail? arXiv:2503.13657, 2025 (v2), 2025.

[23] Ernie W. Chan, Marcel F. Heimlich, Avi Purkayastha, and Robert A. van de Geijn. Collective communication: Theory, practice, and experience. *Concurrency and Computation: Practice and Experience*, 19(13):1749–1783, 2007.

[24] Tushar Deepak Chandra and Sam Toueg. Unreliable failure detectors for reliable distributed systems. *Journal of the ACM*, 43(2):225–267, 1996.

[25] CrewAI. Crewai: Framework for orchestrating role-playing autonomous ai agents. Software documentation, 2024.

[26] David E. Culler, Richard M. Karp, David A. Patterson, Abhijit Sahay, Klaus E. Schauser, Eunice Santos, Ramesh Subramonian, and Thorsten von Eicken. Logp: Towards a realistic model of parallel computation. In *Proc. 4th ACM SIGPLAN Symp. on Principles and Practice of Parallel Programming (PPoPP)*, pages 1–12, 1993.

[27] John T. Daly. A higher order estimate of the optimum checkpoint interval for restart dumps. *Future Generation Computer Systems*, 22(3):303–312, 2006.

[28] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

[29] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. In *Proc. 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 137–150, 2004.

[30] James Demmel, Hong Diep Nguyen, and Peter Ahrens. Reproblas: Reproducible basic linear algebra sub-programs.

[31] James Dinan, Pavan Balaji, Darius Buntinas, David Goodell, William Gropp, and Rajeev Thakur. An implementation and evaluation of the MPI 3.0 one-sided communication interface. *Concurrency and Computation: Practice and Experience*, 28(17):4385–4404, 2016.

[32] Jack J. Dongarra, Steve W. Otto, Marc Snir, and David Walker. A message passing standard for MPP and workstations. *Communications of the ACM*, 39(7):84–90, July 1996. Authoritative on the 1993–94 voting rules, participant counts, funding, and the library/communicator rationale.

[33] Ehtesham, A., et al. A survey of agent interoperability protocols: Model context protocol (mcp), agent communication protocol (acp), agent-to-agent protocol (a2a), and agent network protocol (anp). arXiv:2505.02279, 2025, 2025.

[34] Erman, L. D., Hayes-Roth, F., Lesser, V. R., Reddy, D. R. The hearsay-ii speech-understanding system: Integrating knowledge to resolve uncertainty. *ACM Computing Surveys* 12(2):213–253, 1980, 1980.

[35] Graham E. Fagg and Jack Dongarra. FT-MPI: Fault tolerant MPI, supporting dynamic applications in a dynamic world. *Lecture Notes in Computer Science*, 1908:346–353, 2000.

[36] Finin, T., Fritzson, R., McKay, D., McEntire, R. Kqml as an agent communication language. CIKM 1994, 1994.

[37] FIPA. Fipa acl message structure specification. SC00061G, 2002, 2002.

[38] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2):374–382, 1985.

[39] A2A Project (Linux Foundation). Protocol definition (v1.0 protobuf).

[40] W. (LangChain) Fu-Hinthorn. Benchmarking multi-agent architectures.

[41] Al Geist, Adam Beguelin, Jack Dongarra, Weicheng Jiang, Robert Manchek, and Vaidy Sunderam. *PVM: Parallel Virtual Machine — A Users' Guide and Tutorial for Networked Parallel Computing*. MIT Press, Cambridge, MA, 1994.

[42] Robert Gerstenberger, Maciej Besta, and Torsten Hoefler. Enabling highly-scalable remote memory access programming with MPI-3 one sided. *Scientific Programming*, 22(2):75–91, 2014.

[43] William Gropp, Ewing Lusk, and Deborah Swider. Improving the performance of mpi derived datatypes. In *Proc. Third MPI Developer's and User's Conference (MPIDC'99)*, pages 25–30, 1999. Taxonomy of derived-datatype memory-access patterns; stack-based (dataloop) processing. Page numbers [UNVERIFIED].

[44] Neil J. Gunther. *Guerrilla Capacity Planning: A Tactical Approach to Planning for Highly Scalable Applications and Services*. Springer, 2007.

[45] Carl Hewitt, Peter Bishop, and Richard Steiger. A universal modular actor formalism for artificial intelligence. In *Proc. 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 235–245, 1973.

[46] C. A. R. Hoare. Communicating sequential processes. *Communications of the ACM*, 21(8):666–677, 1978.

[47] Roger W. Hockney. The communication challenge for mpp: Intel paragon and meiko cs-2. *Parallel Computing*, 20(3):389–398, 1994.

[48] Torsten Hoefler. New and old features in mpi-3.0: The past, the standard, and the future, 2012.

[49] Torsten Hoefler, James Dinan, Rajeev Thakur, Brian Barrett, Pavan Balaji, William Gropp, and Keith Underwood. Remote memory access programming in MPI-3. *ACM Transactions on Parallel Computing*, 2(2):9:1–9:26, 2015. Formal axiomatic models because the semantics are "hard to extract from the English prose in the standard".

[50] Torsten Hoefler, Timo Schneider, and Andrew Lumsdaine. Loggopsim – simulating large-scale applications in the loggops model. In *Proc. 19th ACM Int'l Symp. on High Performance Distributed Computing (HPDC), Workshop on Large-Scale System and Application Performance*, pages 597–604, 2010.

[51] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. Metagpt: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.00352.

[52] Kuang-Hua Huang and Jacob A. Abraham. Algorithm-based fault tolerance for matrix operations. *IEEE Transactions on Computers*, C-33(6):518–528, 1984.

[53] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. Gpipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[54] Intel Corporation. oneapi math kernel library developer guide: Obtaining numerically reproducible results / reproducibility conditions (conditional numerical reproducibility), 2025.

[55] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.

[56] Alan H. Karp and Horace P. Flatt. Measuring parallel processor performance. *Communications of the ACM*, 33(5):539–543, 1990.

[57] Laban, P., Hayashi, H., Zhou, Y., Neville, J. Llms get lost in multi-turn conversation. arXiv:2505.06120, 2025; ICLR 2026, 2025.

[58] LangChain. Langgraph: Low-level orchestration framework for stateful agents. Software documentation, 2024.

[59] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for "mind" exploration of large language model society. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.17760.

[60] Lin, C., Han, Z., Zhang, C., Yang, Y., Yang, F., Chen, C., Qiu, L. Parrot: Efficient serving of llm-based applications with semantic variable. OSDI '24, pp. 929–945.

[61] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[62] Grzegorz Malewicz, Matthew H. Austern, Aart J. C. Bik, James C. Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. Pregel: A system for large-scale graph processing. In *Proc. ACM SIGMOD International Conference on Management of Data*, pages 135–146, 2010.

[63] Mei, K., et al. Aios: Llm agent operating system. arXiv:2403.16971, 2024, 2024.

[64] Message Passing Interface Forum. *MPI-3.1 Standard, Matching Probe.*

[65] Message Passing Interface Forum. MPI: A message passing interface. In *Proceedings of the 1993 ACM/IEEE Conference on Supercomputing (SC '93)*, pages 878–883, Portland, Oregon, USA, 1993. ACM.

[66] Message Passing Interface Forum. MPI: A message-passing interface standard. *International Journal of Supercomputer Applications and High Performance Computing*, 8(3/4), 1994. MPI-1.0, released 5 May 1994.

[67] Message Passing Interface Forum. *MPI Standard, Derived Datatypes (typemap and type signature)*, 1995–2023.

[68] Message Passing Interface Forum. MPI-2: Extensions to the message-passing interface. Technical report, University of Tennessee, Knoxville, July 1997. 18 July 1997. Cited for §11.7 Semantics and Correctness (RMA rationale).

[69] Message Passing Interface Forum. MPI: A message-passing interface standard, version 3.1. Technical report, MPI Forum, June 2015. 4 June 2015. Cited for §12.4 MPI and Threads.

[70] Message Passing Interface Forum. MPI: A message-passing interface standard, version 4.0. Technical report, MPI Forum, June 2021. 9 June 2021. Cited for the authoritative version/date table in §1.

[71] Mieczkowski, E. and Collins, K. M. and Sucholutsky, I. and Vélez, N. and Griffiths, T. L. Language model teams as distributed systems. arXiv:2603.12229, 2026.

[72] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging ai applications. In *Proc. 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 561–577, 2018.

[73] Packer, C., et al. Memgpt: Towards llms as operating systems. arXiv:2310.08560, 2023. (now Letta), 2023.

[74] ParaSoft Corporation, Monrovia, CA. *Express User's Guide, Version 3.2.5*, 1992.

[75] A2A Project. Life of a task.

[76] Model Context Protocol. Sep-2322: Multi round-trip requests.

[77] Model Context Protocol. Specification, revision 2026-07-28, 2026.

[78] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. Chatdev: Communicative agents for software development. In *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2307.07924.

[79] Rolf Rabenseifner. Optimization of collective reduction operations. In *Computational Science – ICCS 2004, LNCS 3036*, pages 1–9, 2004.

[80] Chroma Research. Context rot: How increasing input tokens impacts llm performance. Technical report, July 2025, 2025.

[81] Ralf Reussner, Jesper Larsson Träff, and Gunnar Hunzelmann. A benchmark for mpi derived datatypes. In *Recent Advances in PVM and MPI, 7th European PVM/MPI Users' Group Meeting, LNCS 1908*, pages 10–17, 2000. SKaMPI datatype benchmarks; up to 16x penalty on Cray T3E vs 2–4x on NEC SX-5.

[82] Peter Sanders, Jochen Speck, and Jesper Larsson Träff. Two-tree algorithms for full bandwidth broadcast, reduction and scan. *Parallel Computing*, 35(12):581–594, 2009. Conference version: LNCS 4757, pp. 17–26, 2007, doi:10.1007/978-3-540-75416-9_10.

[83] Anthony Skjellum and Alvin P. Leung. Zipcode: A portable multicomputer communication library atop the reactive kernel. In David W. Walker and Quentin F. Stout, editors, *Proceedings of the 5th Distributed Memory Concurrent Computing Conference*, pages 767–776, Charleston, SC, 1990. IEEE Press.

[84] R. G Smith. The contract net protocol: High-level communication and control in a distributed problem solver. *IEEE Transactions on Computers* C-29(12):1104–1113, 1980, 1980.

[85] Marc Snir, Steve Otto, Steven Huss-Lederman, David Walker, and Jack Dongarra. Buffering and safety. In *MPI: The Complete Reference*. MIT Press, 1996. The "throttle effect".

[86] Marc Snir, Steve Otto, Steven Huss-Lederman, David Walker, and Jack Dongarra. Why is MPI so big? In *MPI: The Complete Reference*. MIT Press, 1996. "In all there are 128 MPI routines".

[87] Temporal Technologies. Temporal: Durable execution platform. Software documentation, 2024.

[88] Rajeev Thakur, Rolf Rabenseifner, and William Gropp. Optimization of collective communication operations in mpich. *International Journal of High Performance Computing Applications (IJHPCA)*, 19(1):49–66, 2005.

[89] Jesper Larsson Träff, Rolf Hempel, Hubert Ritzdorf, and Falk Zimmermann. Flattening on the fly: Efficient handling of mpi derived datatypes. In *Recent Advances in PVM and MPI, 6th European PVM/MPI Users' Group Meeting, LNCS 1697*, pages 109–116, 1999.

[90] Träff, J. L. Brief announcement: Improved, partly optimal algorithms for mpi_exscan and mpi_scan. See docs/research/02-mpi-algorithms.md for the retrieved record, 2025. Motivated by reduction operators whose application is expensive, minimising operator applications subject to a round bound.

[91] Leslie G. Valiant. A bridging model for parallel computation. *Communications of the ACM*, 33(8):103–111, 1990.

[92] David W. Walker. Standards for message-passing in a distributed memory environment. Technical Report ORNL/TM-12147, Oak Ridge National Laboratory, August 1992. Report of the First CRPC Workshop on Standards for Message Passing in a Distributed Memory Environment, Williamsburg, Virginia, April 29–30, 1992; 68 attendees.

[93] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation. In *COLM*, 2024. arXiv:2308.08155.

[94] Wu, M., et al. Transagents: Build your translation company with language agents. EMNLP 2024 System Demonstrations (2024.emnlp-demo.14), 2024.

[95] Wu, M., Yuan, Y., Haffari, G., Wang, L. (perhaps) beyond human translation: Harnessing multi-agent collaboration for translating ultra-long literary texts. *TACL* 2025 (2025.tacl-1.42); arXiv:2405.11804, 2025.

[96] W. (Cognition) Yan. Don't build multi-agents. 2025, 2025.

[97] et al Yang. A survey of ai agent protocols. arXiv:2504.16736, 2025 (v3), 2025.

[98] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *Proc. 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 15–28, 2012.